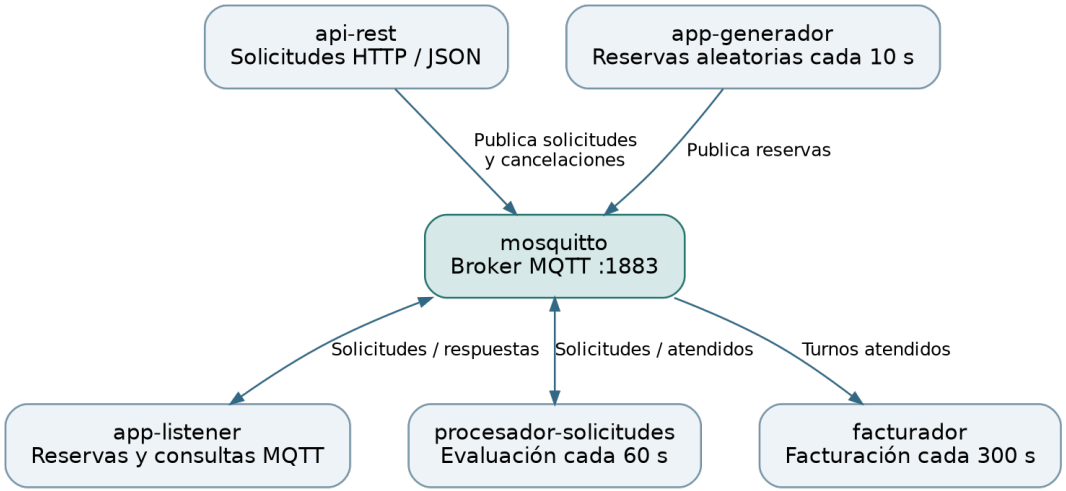
# Componentes y arquitectura

Sistema de reserva de turnos | Producto final - Parte 6

La solución se despliega con Docker Compose y combina una API REST, procesos Java independientes, mensajería MQTT y una base MySQL compartida. El generador permite simular clientes; la API permite operar el sistema mediante HTTP. Los diagramas describen la implementación entregada y las configuraciones de intervalos del Compose.

## 1. Comunicación entre componentes mediante MQTT

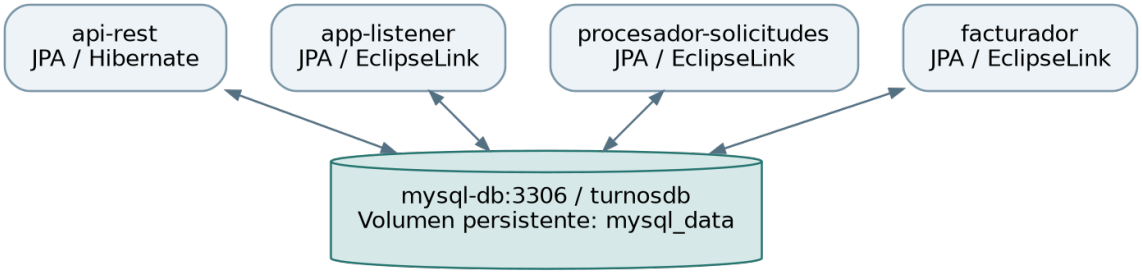



Las flechas indican quién publica y quién recibe. Las conexiones de base de datos se muestran por separado en la página siguiente. La API recibe HTTP desde curl, Swagger UI u otros clientes externos.

| Tópico | Publicador | Suscriptor / uso |
|---|---|---|
| turnos/reserva | app-generador o cliente MQTT | app-listener: recibe datos de una reserva. |
| turnos/solicitudes | api-rest | procesador-solicitudes: recibe un aviso con reservaId. |
| turnos/cancelar | api-rest o cliente MQTT | app-listener: procesa la cancelación. |
| turnos/agenda y turnos/disponibilidad | Cliente MQTT | app-listener: consulta y publica en el tópico correspondiente terminado en /respuesta. |
| turnos/atendidos | procesador-solicitudes | facturador: recibe un aviso con reservaId. |

# Persistencia y despliegue

## 2. Acceso a la base de datos compartida




Las flechas representan lectura y escritura mediante JDBC/JPA. El generador y Mosquitto no acceden a MySQL. Los procesos comparten el esquema turnosdb; cada uno se ejecuta en su propio contenedor. La API usa Hibernate y los procesos del proyecto principal usan EclipseLink.

| Servicio de Compose | Responsabilidad | Acceso desde la computadora |
|---|---|---|
| api-rest | CRUD de establecimientos y personal; solicitudes, cancelación, agenda y disponibilidad de reservas. | http://localhost:8080/api |
| app-generador | Publica datos de reservas simuladas cada 10 segundos. | Sin puerto HTTP. |
| app-listener | Consume reservas y cancelaciones; responde consultas MQTT de agenda y disponibilidad. | Mediante el broker MQTT. |
| procesador-solicitudes | Valida solicitudes, asigna o rechaza; marca turnos como atendidos según su fecha y hora. | Sin puerto HTTP; ciclo de 60 segundos. |
| facturador | Procesa turnos atendidos y registra cliente, factura e ítem de factura. | Sin puerto HTTP; ciclo de 300 segundos. |
| mosquitto | Distribuye mensajes MQTT entre publicadores y suscriptores. | localhost:1883 |
| mysql-db | Almacena establecimientos, personal, reservas, clientes, facturas e ítems. | localhost:3307 (internamente mysql-db:3306). |

## 3. Organización del despliegue

Compose crea una red interna donde se resuelven los nombres de los servicios. Los contenedores se conectan a tcp://mosquitto:1883 y a mysql-db:3306. El puerto 3307 se utiliza desde herramientas externas, como MySQL Workbench. Los datos de MySQL se conservan en el volumen mysql_data; la configuración, los datos y los registros de Mosquitto se montan desde sus carpetas del proyecto.

Maven compila y empaqueta los módulos. Docker construye las imágenes de las aplicaciones Java con sus Dockerfiles. La API se ejecuta como JAR de Spring Boot; los otros procesos seleccionan su clase principal mediante MAIN_CLASS. Los nombres anteriores corresponden a servicios, independientemente del nombre asignado a cada contenedor.

# Flujos de funcionamiento

## 4. Reserva desde la API REST

**1.** Un cliente envía POST /api/reservas con los datos del solicitante, el establecimiento, el personal y la fecha y hora del turno.
**2.** La API persiste la solicitud con estado SOLICITADO, publica un aviso en turnos/solicitudes y responde HTTP 202 con la reserva.
**3.** En su ciclo, el procesador consulta las solicitudes, valida los datos y el horario y asigna AGENDADO o un estado de rechazo.
**4.** El cliente consulta GET /api/reservas/{id} para conocer el resultado. HTTP 202 indica aceptación para procesamiento; la confirmación del turno se obtiene al observar AGENDADO.

## 5. Reserva desde el generador MQTT

El generador publica el contenido de la reserva en turnos/reserva. El listener recibe el JSON, realiza las comprobaciones de esa vía y persiste la reserva. El procesador evalúa las solicitudes pendientes en MySQL. Los identificadores del personal y del establecimiento solicitados se guardan para permitir esta evaluación posterior.

## 6. Cancelación, agenda y disponibilidad

DELETE /api/reservas/{id} publica una solicitud en turnos/cancelar. El listener procesa la operación y cambia el estado de la reserva a CANCELADO cuando corresponde. Se consulta nuevamente la reserva para verificar el resultado. Las consultas HTTP de agenda y disponibilidad acceden a la base desde la API; las consultas MQTT se envían a turnos/agenda y turnos/disponibilidad y sus respuestas se publican en los tópicos /respuesta respectivos.

## 7. Atención y facturación automáticas

El procesador revisa los turnos AGENDADO y marca como ATENDIDO aquellos que cumplen la condición temporal implementada. Después del commit publica sus identificadores en turnos/atendidos. El facturador procesa los turnos ATENDIDO, busca o crea el cliente por correo, agrupa la facturación por cliente, año y mes y registra el ítem de la reserva. La reserva pasa a FACTURADO. Los ciclos también consultan pendientes en MySQL para recuperar trabajo que no haya sido tomado desde los avisos MQTT.

## 8. Alcance y consideraciones de integración

La implementación usa una base compartida: el desacoplamiento entre procesos mediante MQTT convive con el acceso a las mismas tablas. Persistir una operación y publicar su aviso son pasos separados; la solución no implementa una transacción distribuida entre MySQL y Mosquitto. Los procesos periódicos permiten recuperar solicitudes y turnos atendidos desde la base. La restricción única de ITEMFACTURA.reserva_id ayuda a evitar la doble facturación de una reserva.

La arquitectura de facturación descrita surge del código del producto. Su verificación funcional requiere observar un turno ATENDIDO que termine en FACTURADO y sus registros de factura e ítem; las pruebas de creación y cancelación de reservas no verifican por sí solas ese recorrido.

Base de elaboración: docker-compose.yml, Dockerfiles, configuración JPA y clases API, SubscriberReservaMQTT, GeneradorReservasMQTT, ProcesadorSolicitudes, FacturadorAutomatico y EventosTurnosMQTT del proyecto proporcionado; se consideran los ajustes de integración comprobados durante las pruebas de reservas.